

Machine Learning

Data Preprocessing



Dr. G. Omprakash

KLEF Hyderabad (Aziz Nagar) – Dept. of ECE

Machine Learning (25SC2107E)

Session Outcome

- Understand why raw data cannot be given directly to a learning algorithm, and apply *scaling*, *encoding* and *imputation* to prepare it.

Data Preprocessing

Raw Data Cannot Be Used Directly

A real PlacementPredict record looks like this:

ID	CGPA	Aptitude	Stream	Skill level	Salary expected	Placed
S101	8.4	78	ECE	High	6,00,000	1
S102	—	65	CSE	Medium	4,50,000	1
S103	6.1	52	MECH	Low	3,00,000	0
S104	7.2	—	CSE	High	—	1

- **“ECE” is a word.** $\theta^T \mathbf{x}$ cannot multiply a word by a number.
- **“High” is an order**– have a ranking–Convert it into number.
- **The blanks are missing values.** Any arithmetic with a blank gives a blank.
- **The scales are wildly different:** CGPA is 0–10, salary is in lakhs.

Attribute

ATTRIBUTE VS FEATURE

They are **related**, but **not the same**!

ATTRIBUTE

An attribute is any **property** or **characteristic** that describes an entity in a dataset.

- Broader term.
- Can be input (features) or output (label/target).
- All columns in a dataset are **attributes**.

Example – House Dataset



Attribute (Column)	What it describes
Area	Size of the house
Bedrooms	Number of bedrooms
Bathrooms	Number of bathrooms
Age	Age of the house
Price	Market price of the house



In this dataset, **ALL columns** (Area, Bedrooms, Bathrooms, Age, Price) are **attributes**.

FEATURE

A feature is an **input attribute** used by the model to make predictions.

- Narrower term.
- Only the **input attributes** (predictors) are **features**.
- Do not include the output (label/target).

Example – Predict House Price

Features (Inputs)

- Area
- Bedrooms
- Bathrooms
- Age



Label (Output)



Here, Area, Bedrooms, Bathrooms, Age are **features** (inputs). Price is the **label** (output).

Aspect	Attribute	Feature
Meaning	Any property/characteristic of data	Input attribute used for prediction
Scope	Includes all columns (inputs + output)	Includes only input columns
Role in ML	Can be input or output	Always an input (predictor)
Example (House Price)	Area, Bedrooms, Bathrooms, Age, Price	Area, Bedrooms, Bathrooms, Age



In Short

All **features** are **attributes**,
but **not all attributes**
are **features**.



Key Takeaway: **Attributes** describe the data. **Features** are the attributes you use to train your model.

Attribute

CAN I TELL IMPORTANT ATTRIBUTES AS FEATURES?

Yes—But Only If They Are Used as **input** to the Model!

1. ATTRIBUTE

Any property (column) in a dataset.
It can be input, output, or just information.



All columns
are **attributes**



Describes an
entity

2. FEATURE

An attribute that is used as an
input to a machine learning model
to make predictions.



Used by the model
as **input** (predictors)



Helps the model
learn patterns



Rule to Remember: All features are **attributes**,
but not all attributes are **features**.



Important attributes can be
called **features** only if they are
selected as **inputs** for prediction.

EXAMPLE: PREDICT HOUSE PRICE

Dataset

Area (sq ft)	Bedrooms	Age of House (Years)	Owner Name	Price (₹ Lakhs)
1200	2	5	Rahul	45
1800	3	2	Priya	70
1500	2	10	Amit	50
2000	4	1	Neha	80

FEATURES (Inputs)



LABEL (Output)



ATTRIBUTES IN THIS DATASET



Area



Bedrooms



Age of House



Owner Name



Price

Total Attributes = 5

FEATURES USED FOR PREDICTION



Area



Bedrooms



Age of House

Total Features = 3

NOT FEATURES



Owner Name
(Not used for
prediction)



Price
(Output/Label,
not an input)



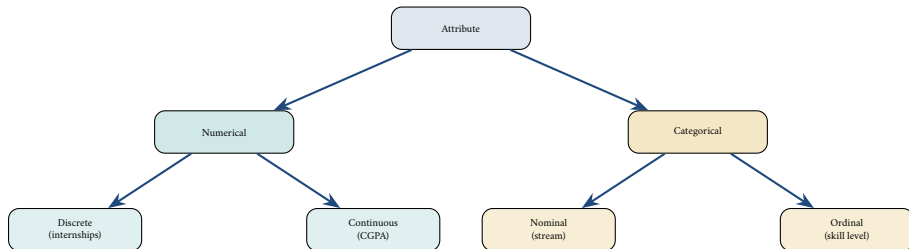
In this example:

- All columns (5) are **attributes**.
- Area, Bedrooms, Age are important attributes and are used as **inputs** → so they are **features**.
- Owner Name is an attribute but not used for prediction → **not a feature**.
- Price is the attribute we want to predict → it is the **label (output)**.

GOOD CLASSROOM STATEMENT

"Features are the
important **input attributes**
that the model uses to
make **predictions**."

Types of Attributes



Type	Example	Order?	Treatment
Continuous	CGPA, aptitude	yes	scaling
Discrete	internships, projects	yes	scaling (often as is)
Nominal	stream: CSE/ECE/MECH	no	one-hot / target encoding
Ordinal	skill: Low/Medium/High	yes	ordinal encoding
Missing	blank cells	—	imputation + indicator

Why Scaling Is Needed

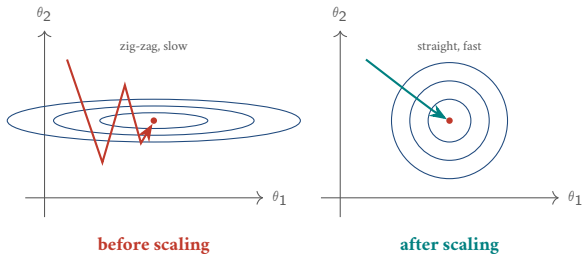


Figure: Contours of the cost, and the path of gradient descent

- **Gradient descent:** one learning rate η is used for all directions. If one feature is in thousands and another in units, no single η suits both.
- **Regularisation** The penalty charges by the size of θ_j , so an unscaled feature is punished unfairly.
- **Distance-based methods** (KNN, SVM, K-means): the feature with the largest numbers dominates the distance.
- *Not needed* for decision trees and random forests, which only compare values within a column.

StandardScaler (Standardisation)

Every feature is shifted to mean 0 and scaled to standard deviation 1:

$$x' = \frac{x - \mu}{\sigma} \quad \text{where} \quad \mu = \frac{1}{m} \sum_{i=1}^m x^{(i)}, \quad \sigma = \sqrt{\frac{1}{m} \sum_{i=1}^m (x^{(i)} - \mu)^2}$$

Example. CGPA values 6.0, 7.0, 8.0, 9.0. $\mu = 7.5$, $\sigma = \sqrt{1.25} = 1.1180$.

x	$x - \mu$	$x' = (x - \mu)/\sigma$
6.0	-1.5	-1.3416
7.0	-0.5	-0.4472
8.0	+0.5	+0.4472
9.0	+1.5	+1.3416

- The result is **not** bounded — an outlier can still give $x' = 5$.
- The shape of the distribution is unchanged; only its centre and spread move.
- Default choice for linear models, logistic regression, SVM and neural networks.

STANDARDIZATION | Bring Data to a Common Scale

WHAT IS STANDARDIZATION?

Standardization (Z-score Scaling) transforms data so that:

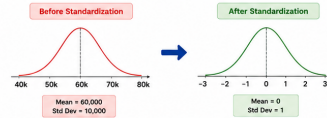
- ✓ Mean of the transformed data = 0
- ✓ Standard deviation = 1

FORMULA

$$Z = \frac{x - \mu}{\sigma}$$

x = original value
 μ = mean of the data
 σ = standard deviation of the data
 z = standardized value

EXPLAINED USING BELL CURVE



All values are transformed so the center is 0, and spread is measured in standard deviations.

EXAMPLE: SALARY DATA BEFORE AND AFTER STANDARDIZATION

BEFORE STANDARDIZATION

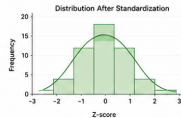
Employee	Salary (₹)
A	45000
B	55000
C	65000
D	75000
E	85000

Mean (μ) = 65,000
 Std Dev (σ) = 14,142.14

AFTER STANDARDIZATION (Z-SCORES)

Employee	Standardized Salary (z)
A	-1.41
B	-0.71
C	0.00
D	0.71
E	1.41

Mean (μ) = 0
 Std Dev (σ) = 1



TIP: Always standardize features when using algorithms that assume a normal distribution or are sensitive to the scale of data (e.g., SVM, Linear Regression, PCA, K-Means).

PYTHON IMPLEMENTATION

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
df[['Salary']] = scaler.fit_transform(df[['Salary']])
```

This transforms the 'Salary' column to have mean = 0 and std dev = 1.

NORMALIZATION vs STANDARDIZATION

Feature	Normalization (Min-Max Scaling)	Standardization (Z-score Scaling)
Range	Scaled to 0 - 1	Mean = 0, Std Dev = 1
Sensitive to Outliers	Yes, very sensitive	Less sensitive
Effect of Scale	Changes shape with outliers	Maintains shape better
Best Used For	Neural Networks, Deep Learning	Regression, SVM, PCA, Clustering

WHERE TO USE?



Neural Networks
 Standardization helps in faster convergence.



Regression
 Standardization helps models assume normal distribution.



SVM
 SVM works better when features are on the same scale.



PCA / Clustering
 Distance-based algorithms benefit from standardized data.

MinMaxScaler (Normalisation)

Every feature is squeezed into a fixed interval, usually $[0, 1]$:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

Example. The same CGPA values. $x_{\min} = 6.0$, $x_{\max} = 9.0$, range = 3.0.

x	$x - x_{\min}$	x'
6.0	0.0	0.0000
7.0	1.0	0.3333
8.0	2.0	0.6667
9.0	3.0	1.0000

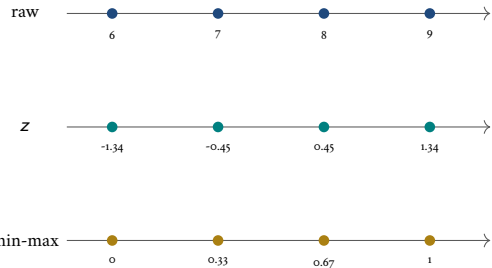


Figure: Only the axis labels change; the spacing is preserved

Normalization (Min-Max Scaling)



Normalization scales all numerical values to a fixed range [0, 1].
It is useful when features have **different units** or **ranges**.

1. Problem

Features are on different scales.

	Age (years)	20
	Salary (₹)	900000
	Marks (%)	80

Machine learning models may give more importance to features with larger values.

2. What is Normalization?

Normalization scales values between 0 and 1.
All features will have the same scale.



Helps in faster convergence of models and improves performance.

3. Formula (Min-Max Scaling)

Each value is transformed using:

$$x_{\text{norm}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

x = Original value
 $x_{\min}$ = Minimum value in the feature
 $x_{\max}$ = Maximum value in the feature
 x_{norm} = Normalized value (between 0 and 1)

4. Example

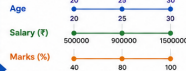
Feature: Age

Original Age (x)	20	25	30
Calculation	$\frac{20-20}{30-20} = 0$	$\frac{25-20}{30-20} = 0.5$	$\frac{30-20}{30-20} = 1$
Normalized Age (x_{norm})	0	0.5	1

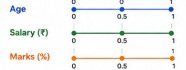


Now all values are between 0 and 1.
This removes the effect of different scales.

Before Normalization



After Normalization (0 to 1)



5. Python Implementation

```
from sklearn.preprocessing import MinMaxScaler
import pandas as pd

# Sample Data
data = {
    'Age': [20, 25, 30],
    'Salary': [500000, 900000, 1500000],
    'Marks': [40, 80, 100]
}
df = pd.DataFrame(data)

# Apply Normalization
scaler = MinMaxScaler()
df[['Age', 'Salary']] = scaler.fit_transform(
    df[['Age', 'Salary']]
)

# Result
print(df)
```



Key Takeaway



Normalization scales values to [0, 1].



It handles different units and scales.



Improves model performance and convergence speed.



Useful for algorithms like KNN, SVM, Neural Networks.

Which Scaler to Use?

	StandardScaler	MinMaxScaler	RobustScaler
Formula	$\frac{x - \mu}{\sigma}$	$\frac{x - x_{\min}}{x_{\max} - x_{\min}}$	$\frac{x - \text{median}}{IQR}$
Output range	unbounded	[0, 1]	unbounded
Centre	mean = 0	min = 0	median = 0
Effect of outliers	moderate	severe	small
Typical use	linear, logistic, SVM, NN	image pixels, bounded inputs	data with outliers

Ordinal Encoding

Used when the categories have a **natural order**.

Skill level	Encoded
Low	0
Medium	1
High	2

- One column only — compact.
- The numbers carry the meaning “High > Medium > Low”.
- It also implies that the *gap* Low→Medium equals the gap Medium→High, which may not be true.

Note: never use ordinal encoding for unordered categories.

If we write CSE = 0, ECE = 1, MECH = 2, then the model believes

$$\text{MECH} - \text{ECE} = \text{ECE} - \text{CSE} \quad \text{and} \quad \text{ECE} = \frac{\text{CSE} + \text{MECH}}{2}$$

One-Hot Encoding

Used for **nominal** (unordered) categories. One new column per category; exactly one of them is 1.

Stream	is_CSE	is_ECE	is_MECH
CSE	1	0	0
ECE	0	1	0
MECH	0	0	1
ECE	0	1	0

- No false ordering is created — all three categories are at equal distance from each other.
- Each category gets its own parameter θ_j , so the model can learn a separate effect for each stream.

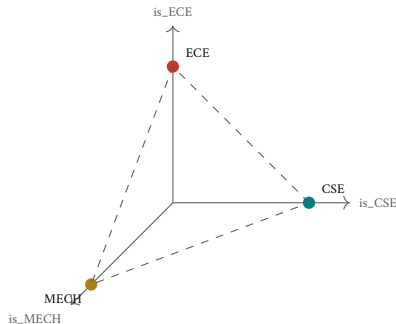


Figure: The three streams sit at equal distances

Drawbacks: one column per category — 500 colleges give 500 mostly-zero columns. The columns also add up to 1, making $\mathbf{X}^T \mathbf{X}$ singular;

Missing Values

Why values go missing:

- The student did not fill the form.
- The field does not apply (no internship $\Rightarrow$ no stipend).
- A sensor or a data-entry step failed.

Why we cannot simply delete the rows:

- If 30% of students have one blank each, deleting rows may throw away most of the data.
- Missing-ness is often **not random**. Students with low CGPA are the ones who leave it blank — deleting them biases the model.

students $\times$ features

			?	
?				
				?
		?		
	?			
			?	

Figure: Blanks scattered over the table

Question: with what number should a blank CGPA be filled, and does the *fact* that it was blank itself carry information?

Simple Imputation: Mean and Median

Replace every blank in a column by a single value computed from that column. **Example.** CGPA column: 6.0, 7.0, 8.0, 9.0, blank, 40.0 (the 40.0 is a typing error for 4.0).

Method	Value used	Comment
Mean	$\frac{6 + 7 + 8 + 9 + 40}{5} = 14.0$	dragged far up by the outlier
Median	middle of 6, 7, 8, 9, 40 $\Rightarrow$ 8.0	unaffected by the outlier
Mode	most frequent value	used for categorical columns
Constant	-1 or "Unknown"	marks the blank explicitly

- Use the **median** when the column is skewed or has outliers; the mean when it is roughly symmetric.
- Imputation reduces the natural spread of the column, because many rows now share one identical value.
- `scikit-learn: SimpleImputer(strategy='mean' | 'median' | 'most_frequent' | 'constant')`.

MissingIndicator: the Blank Itself Is Information

Add one extra 0/1 column recording *whether* the value was missing, and then impute as usual.

Student	CGPA (raw)	CGPA (imputed)	CGPA_was_missing
S101	8.4	8.4	0
S102	—	7.5	1
S103	6.1	6.1	0
S104	—	7.5	1

- The model can now learn a separate effect for “this student did not report a CGPA”.
- If weak students are the ones who leave the field blank, this indicator can be one of the most useful features in the table.
- Without it, the imputed 7.5 is indistinguishable from a genuine 7.5 and the information is lost forever.
- `scikit-learn: MissingIndicator`, or `SimpleImputer(add_indicator=True)`.

Putting It All Together

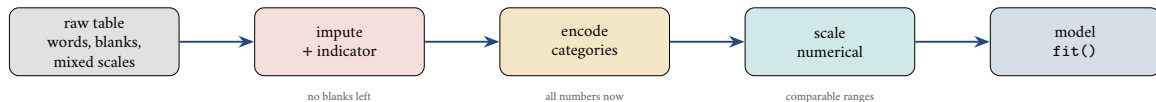


Figure: The order matters: impute, then encode, then scale

- **Impute first** — encoders and scalers cannot work with blanks.
- **Encode next** — scaling can only be applied to numbers.
- **Scale last** — scale the one-hot columns only if the algorithm needs it.
- The whole chain is one Pipeline object, fitted once on the training data and stored with the model.

Summary — Data Preprocessing

- Raw data contains **words**, **blanks** and **mixed scales**; none of these can enter $\theta^T \mathbf{x}$ directly.
- Attributes are numerical (discrete/continuous), categorical **nominal** or **ordinal**, or missing — and each type needs a different treatment.
- **Scaling**: StandardScaler $(x - \mu)/\sigma$ for most models; MinMaxScaler for a bounded range;
- **Encoding**: ordinal for ordered categories; one-hot for unordered ones;
- **Missing values**: mean or median for simple cases,
- Every transformer is **fitted on the training set only** and re-used unchanged at serving time.